



Semester: 2

Subject : Data Structures Using JAVA

Chapter : Application of Tree

Java Program to Perform Insertion, Deletion (All Cases), Searching, Finding Minimum, Maximum, Inorder Successor, Inorder Predecessor, and Inorder Traversal in a Binary Search Tree (BST)

```
class Node {
    int key;
    Node left, right;

    Node(int item) {
        key = item;
        left = right = null;
    }
}

class BST {
    Node root;

    void insert(int key) {
        root = insertRec(root, key);
    }

    Node insertRec(Node root, int key) {
        if (root == null)
            return new Node(key);

        if (key < root.key)
            root.left = insertRec(root.left, key);
        else if (key > root.key)
            root.right = insertRec(root.right, key);
        else
            System.out.println(key + " already exists. Duplicate not allowed.");
    }
}
```



Semester: 2

Subject : Data Structures Using JAVA

Chapter : Application of Tree

```
    return root;
}

boolean search(int key) {
    return searchRec(root, key);
}

boolean searchRec(Node root, int key) {
    if (root == null)
        return false;

    if (root.key == key)
        return true;
    else if (key < root.key)
        return searchRec(root.left, key);
    else
        return searchRec(root.right, key);
}

void delete(int key) {
    if (!search(key)) {
        System.out.println(key + " not found. Deletion not possible.");
    } else {
        root = deleteRec(root, key);
        System.out.println(key + " deleted successfully.");
    }
}

Node deleteRec(Node root, int key) {
    if (root == null)
        return null;
```



Semester: 2

Subject : Data Structures Using JAVA

Chapter : Application of Tree

```
if (key < root.key) {
    root.left = deleteRec(root.left, key);
}
else if (key > root.key) {
    root.right = deleteRec(root.right, key);
}
else {
    // Case 1: Leaf node
    if (root.left == null && root.right == null)
        return null;

    // Case 2: One child
    else if (root.left == null)
        return root.right;

    else if (root.right == null)
        return root.left;

    // Case 3: Two children
    else {
        root.key = minValue(root.right);
        root.right = deleteRec(root.right, root.key);
    }
}
return root;
}

int minValue(Node root) {
    if (root == null) {
        System.out.println("Tree is empty.");
        return -1;
    }
}
```



Semester: 2

Subject : Data Structures Using JAVA

Chapter : Application of Tree

```
}
```

```
while (root.left != null)
    root = root.left;
```

```
return root.key;
```

```
}
```

```
int maxVal(Node root) {
    if (root == null) {
        System.out.println("Tree is empty.");
        return -1;
    }
```

```
while (root.right != null)
    root = root.right;
```

```
return root.key;
```

```
}
```

```
Node findPredecessor(int key) {
    if (!search(key))
        return null;
```

```
Node current = root;
```

```
Node predecessor = null;
```

```
while (current != null) {
    if (key > current.key) {
        predecessor = current;
        current = current.right;
    } else {
```



Semester: 2

Subject : Data Structures Using JAVA

Chapter : Application of Tree

```
        current = current.left;
    }
}
return predecessor;
}
```

```
Node findSuccessor(int key) {
    if (!search(key))
        return null;
```

```
    Node current = root;
    Node successor = null;
```

```
    while (current != null) {
        if (key < current.key) {
            successor = current;
            current = current.left;
        } else {
            current = current.right;
        }
    }
    return successor;
}
```

```
void inorder(Node root) {
    if (root != null) {
        inorder(root.left);
        System.out.print(root.key + " ");
        inorder(root.right);
    }
}
```



Semester: 2

Subject : Data Structures Using JAVA

Chapter : Application of Tree

```
public static void main(String[] args) {  
    BST tree = new BST();  
  
    tree.insert(50);  
    tree.insert(30);  
    tree.insert(70);  
    tree.insert(20);  
    tree.insert(40);  
    tree.insert(60);  
    tree.insert(80);  
  
    System.out.println("Inorder Traversal:");  
    tree.inorder(tree.root);  
  
    System.out.println("\n\nSearch 40: " + tree.search(40));  
    System.out.println("Search 100: " + tree.search(100));  
  
    System.out.println("\nMin Value: " + tree.minValue(tree.root));  
    System.out.println("Max Value: " + tree.maxValue(tree.root));  
  
    tree.delete(20); // leaf node  
    tree.delete(30); // one child  
    tree.delete(50); // two children  
    tree.delete(100); // not found  
  
    System.out.println("\nFinal Inorder Traversal:");  
    tree.inorder(tree.root);  
  
    Node pred = tree.findPredecessor(60);  
    Node succ = tree.findSuccessor(60);
```



Semester: 2

Subject : Data Structures Using JAVA

Chapter : Application of Tree

```
System.out.println("\n\nPredecessor of 60: " + (pred != null ? pred.key :  
"None"));  
System.out.println("Successor of 60: " + (succ != null ? succ.key :  
"None"));  
}  
}
```

Jay Chauhan